

How to write an amazing article

Adam Herout*

Abstract

What is the problem? What is the topic? The aim of this paper? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. How is the problem solved, the aim achieved (methodology)? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. Curabitur massa neque, commodo posuere fringilla ut, cursus at dui. Nulla quis purus a justo pellentesque. What are the specific results? How well is the problem solved? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod. Mauris sed lectus non massa molestie congue. In hac habitasse platea dictumst. So what? How useful is this to Science and to the reader? Lorem ipsum dolor sit amet, consectetur adipiscing elit. Fusce ullamcorper suscipit euismod.

Keywords: Keyword1 — Keyword2 — Keyword3

Supplementary Material: [Demonstation video](#) — [Downloadable code](#)

*herout@fit.vut.cz, Faculty of Information Technology, Brno University of Technology

1. Introduction

With the rise of Large Language Models (LLMs) and the increase in their capabilities, companies are motivated to use LLMs for automation tasks, including testing and the tooling for it. However, LLMs are not plug-and-play; specialized tooling is often required to produce reliable outputs. This work defines a formal base into which LLMs rewrite requirements and test cases written in natural language, so that the result is parsable by test coverage checking systems.

There is an increasing demand to instrument systems in natural language, including the specification of requirements and their test cases for testing systems and system components. Even when requirements and test cases are written in English, they must still be rewritten into a mathematical formal base so that existing test-coverage techniques can be applied. A formal base into which LLMs can convert human-written text is therefore needed. This formal base must be able to express all motivational requirement examples. Its semantics must be easily interpretable by LLMs and

agentic systems, so that agents can rewrite requirements and test case descriptions from human language into the formal base with high accuracy and without changing the semantics; the formalism itself must be unambiguous. It must also be convertible to existing formal bases used by formal verification tools. A good formalism should meet all of the mentioned criteria.

2. Existing methods of requirement formalisation – State of the Art

The translation from natural-language (NL) requirements to formal specifications has long been pursued through controlled-language frontends that constrain the input enough to make it unambiguous and further processable. The recent maturation of large language models has reopened the question of to what degree this translation can be automated without such restrictions, leading to LLM-based pipelines and hybrid approaches that pair LLMs with formal provers and solvers — the rest of this section surveys that landscape. This overview is informed by the recent short

survey of Beg et al. (2025) [1], which classifies the LLM-based literature in detail. It is further complemented with controlled-language frontends that the survey only briefly covers. Table 1 summarises the systems discussed in this section.

Controlled-language frontends. The longest-lived line of work attacks the problem at the *input* side by constraining the natural language itself. Nowakowski et al. (2013) [9] developed the *Requirements Specification Language* and the *ReDSeeDS* platform, which couples a constrained-NL requirements notation with automated transformations into code. Ghosh et al. (2016) [5] present *ARSENAL*, an NLP-based framework that extracts formal requirement specifications from natural language with automatic verification. NASA’s *FRET*, introduced by Giannakopoulou et al. (2020) [6], lets engineers write requirements in *FRETISH*, a restricted English with unambiguous semantics that maps deterministically to temporal logic.

LLM-only pipelines. With the rise of large language models, a growing body of work delegates the full translation to the LLM itself. Nayak et al. (2022) [8] present *Req2Spec*, an NLP pipeline that maps natural-language requirements onto the pattern-based formal specifications consumed by HANFOR; on 222 BOSCH automotive requirements it correctly formalises 71%. Cosler et al. (2023) [2] propose *nl2spec*, which uses few-shot prompting of Codex (code-davinci-002) and Bloom-176B to translate unstructured natural language into Linear-time Temporal Logic (LTL), with iterative user refinement of subformulas. Fang et al. (2024) [3] introduce *AssertLLM*, a three-stage pipeline of customised GPT-4o models (the final stage retrieval-augmented) that generates SystemVerilog Assertions from hardware design specifications with 89% correctness.

Hybrid LLM + prover systems. The most recent strand pairs an LLM with a formal prover or solver that has the final say on correctness. Jiang et al. (2022) [7] introduce *Thor*, which couples a 700M-parameter decoder-only transformer with the Isabelle/HOL theorem prover via Sledgehammer-based premise selection (“Hammers”), raising proof success on PISA from 39% to 57%. Yang et al. (2023) [11] release *LeanDojo* and the retrieval-augmented prover *ReProver*, a ByT5 encoder-decoder paired with premise retrieval over the Lean mathematics library and benchmarked on 98 734 theorems. Quan et al. (2024) [10] propose *ExplanationRefiner*, a refinement loop between an LLM (GPT-4, GPT-3.5-turbo, Llama2-70b, Mixtral-8x7b, or Mistral-small) and the Isabelle/HOL prover that validates and

Table 1. State-of-the-art systems for natural-language to formal-specification translation discussed in this section, in chronological order within each category. Abbreviations: HITL = human-in-the-loop; NLP = natural language processing; MDD = model-driven development; SAL = Symbolic Analysis Laboratory; LTL = linear-time temporal logic; HANFOR = an industrial requirements-analysis tool that consumes pattern-based formal specifications; SVA = SystemVerilog Assertions; HOL = higher-order logic (as in the Isabelle/HOL theorem prover); SMT = satisfiability modulo theories.

Ref.	System	Year	Models	Target	HITL
<i>Controlled-language frontends</i>					
[9]	RSL/ReDSeeDS	2013	— (rule-based)	Code (MDD)	Yes
[5]	ARSENAL	2016	NLP pipeline	SAL/LTL	No
[6]	FRET	2020	— (rule-based)	LTL	Yes
<i>LLM-only pipelines</i>					
[8]	Req2Spec	2022	NLP pipeline	HANFOR	No
[2]	nl2spec	2023	Codex, Bloom-176B	LTL	Yes
[3]	AssertLLM	2024	GPT-4o	SVA	No
<i>Hybrid LLM + prover systems</i>					
[7]	Thor	2022	700M trans-former	Isabelle/HOL	No
[11]	LeanDojo	2023	ByT5 + retrieval	Lean	No
[10]	ExplanationRefiner	2024	GPT-4/3.5, Llama2-70b, Mixtral-8x7b, Mistral-small	Isabelle/HOL	No
[4]	SAT-LLM	2024	LLM + SMT solver	SMT	No

corrects natural-language explanations. Fazelnia et al. (2024) [4] build *SAT-LLM*, which combines an LLM with an SMT solver to detect conflicting requirements at F1 0.91.

3. Contributing by designing a “close-to-NL” formalism

While prior controlled-language frontends [6, 9, 5] and LLM-based translators [2] target a single fixed logic — typically LTL or a tool-specific pattern language — none of the systems surveyed in Section 2 position their formalism around predicate-level matchability between requirements and test cases for coverage-gap detection, nor as a logic-agnostic intermediate explicitly designed to be the translation target for LLMs. The formalism presented in this section is designed around precisely these two properties.

3.1 Formalism requirements and its design

The formalism must satisfy the properties listed below. Each property is paired with a concrete use case expressed in abstract placeholders (signals *A*, *B*; events

92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110

111 e_A, e_B ; etc.) and a brief note on the construct the
112 formalism provides to address it.

113 **Meta-level properties. Logic-agnostic representa-**
114 **tion of system behaviour in time.** The formalism
115 shall not commit to a specific underlying logic, so
116 that a single requirement representation can be retar-
117 geted to different verification back-ends. *Solution:* A
118 requirement is represented as a composition of two
119 kinds of building blocks: (i) ordinary expressions and
120 predicates that, when evaluated against a trace (the
121 same kind of object a test case produces), yield con-
122 crete values or truth values at each point in time; and
123 (ii) temporal predicates that aggregate those point-in-
124 time predicates into a single truth value over the entire
125 trace. The semantics of each building block is defined
126 directly in set-theoretic terms over traces and time,
127 independent of any target back-end. Every temporal
128 predicate has a direct LTL (or metric-LTL) equivalent
129 and a corresponding timed-automaton construction.

130 **Mechanical coverage-gap detection.** Given a set
131 of formalised requirements and a set of formalised test
132 cases, the formalism shall enable a checker to deter-
133 mine which requirement conditions are not exercised
134 by any test case. *Solution:* Requirements and test cases
135 are formalised over the same entities and the same
136 predicates, so that the same sub-formulas appear on
137 both sides. A checker can then evaluate, for each test
138 case, which meaningful combinations of sub-formulas
139 inside a requirement's formula are driven to true and
140 which to false, and from that compute the fraction of
141 relevant combinations that the test suite exercises.

142 **Extensible domains.** Real-world requirements
143 draw on an open-ended range of kinds of things in a
144 system (signals, registers, channels, states, and so on)
145 and of events that can happen to them (a register being
146 written, a channel transmitting a value, a state being
147 entered), and it is unrealistic to fix a closed catalogue
148 up front that covers every domain the formalism may
149 ever be applied to. The formalism shall therefore keep
150 these catalogues extensible. *Example:* To support a
151 new entity kind C representing network packets, only
152 the relevant catalogue is widened; the rest of the for-
153 malism is untouched. *Solution:* The set of entity kinds,
154 the set of value types, the assignment of permitted
155 attributes to each entity kind, and the assignment of
156 permitted events (parameterised by the entity they hap-
157 pen to) to each entity kind are all parameters of the
158 formalism rather than fixed sets. Adding a new entry
159 to any of them requires only registering it in the rele-
160 vant catalogue; existing constructs continue to work
161 unchanged.

Time and values. Discrete and continuous time. 162
The formalism shall support both discrete-time and 163
continuous-time requirements within a single vocabu- 164
lary. *Example:* A discrete-time requirement says “at 165
every step, A is incremented by 3”; a continuous-time 166
requirement says “ e_B must occur within 2 seconds of 167
 e_A ”. *Solution:* The time set T has two flavours: $T_D =$ 168
 $\mathbb{N} \cup \{\infty\}$ for discrete time and $T_C = \mathbb{R}_{\geq 0} \cup \{\infty\}$ for 169
continuous time. Each requirement carries a *flavour* $\in$ 170
 $\{\text{Discrete, Continuous}\}$ field that fixes T for its scope. 171

Entities of multiple kinds carry values, and op- 172
erations modify them in test cases. The formalism 173
shall represent entities of different kinds (registers, 174
states, communication channels, computed signals, 175
etc.) and let each carry a value at any given time 176
point. It shall also provide a way for test cases to de- 177
scribe how those values change over time. *Example:* 178
Register A , state B , and channel C each have a value 179
at every relevant time point; a test case may, for in- 180
stance, write a new value into A , transition B to a new 181
state, or transmit a value through C . *Solution:* The 182
formalism defines an extensible catalogue of entity 183
kinds (currently signal, storage, event, event-trigger, 184
channel, state, value, abstract, and set), and a valua- 185
tion maps each entity to its value at each time point of 186
a trace. Modifications of those values are expressed 187
through *operations* — an extensible catalogue of ac- 188
tions, currently including *calculate*, *write*, *read*, *fire*, 189
set_state, *transmit*, *receive*, *insert*, *remove*, and *clear* 190
— that a test case fires at specific time points to drive 191
the trace. Each entity kind has its own subset of appli- 192
cable operations (e.g. *write* applies to storage, *fire* to 193
event-triggers, *transmit* to channels). 194

Initial-value constraints. The formalism shall be 195
able to constrain the value of an entity at the initial 196
time point. *Example:* “Signal A has initial value 0.” *So-* 197
lution: The temporal construct $\text{Initial}(\psi) \Leftrightarrow \psi(p, 0)$ 198
asserts a point-in-time predicate at time 0. 199

Reference to an entity's value at another time 200
point. The formalism shall allow a constraint to refer 201
to the value an entity had (or will have) at an ear- 202
lier or later time point, enabling recurrences, toggles, 203
and look-ahead conditions. *Example:* “ $A(\text{Now}) =$ 204
 $A(\text{Now} - 1) + 3$ ”; “ B equals the negation of B 's previ- 205
ous value”; or “ C at the next step equals the current 206
value of D ”. *Solution:* $\text{Val}_p(e, t)$ returns the value of 207
 e at any time t , past or future; for discrete time, *Prev*, 208
Next, and *RelTime* shift the time argument backward 209
or forward; $\text{ValBefore}_p(e, t)$ exposes the left limit at t . 210

Static entity-attribute access. The formalism 211
shall expose static attributes of an entity (independent 212
of trace and time). *Example:* The address of register 213

214 A is the constant 0xAA000018 and can be referenced
 215 as such. *Solution*: Static attributes are stored in an en-
 216 tity’s modifier map $\mu_e : \text{ModKey} \rightarrow \text{ModVal}$; accessors
 217 such as $\text{Addr}(e) = \mu_e(\text{address})$ retrieve them without
 218 reference to ρ or t .

219 **Events and triggers. Event occurrence at a given**
 220 **time point.** The formalism shall express that an event
 221 occurs at a particular time point. *Example*: “ e_A is hap-
 222 pening at t .” *Solution*: The predicate $\text{Happening}(e, t)$,
 223 where e is an entity of type `Event`.

224 **Past-event semantics.** The formalism shall ex-
 225 press that an event has occurred at some point up to
 226 the current time. *Example*: “ e_A has happened at least
 227 once.” *Solution*: The predicate $\text{HasHappened}_\rho(e, t)$.

228 **Reference to event-occurrence times.** The for-
 229 malism shall expose the time of a specific past or future
 230 occurrence of an event. *Example*: “the time of the most
 231 recent occurrence of e_A .” *Solution*: $\text{LastOcc}_\rho(ev, t, n)$
 232 returns the interval spanning the n most recent occur-
 233 rences of ev up to t ; FirstOcc_ρ does the same in the
 234 forward direction.

235 **Counting event occurrences within an interval.**
 236 The formalism shall count how many times an event
 237 occurred within a given interval, with control over
 238 whether interval endpoints are included. *Example*:
 239 “the number of occurrences of e_A in the half-open in-
 240 terval $(t_1, t_2]$.” *Solution*: $\text{EvtOccCount}_\rho(ev, i)$ returns
 241 the count.

242 **Conditional and logical structure. Value-conditional**
 243 **branches.** The formalism shall support constraints
 244 that branch on the current value of an entity. *Exam-*
 245 *ple*: “If A is true then B equals X ; otherwise B equals
 246 Y .” *Solution*: The comparison predicate Cmp and the
 247 implication predicate $\text{Implies}(A, B)$ together express
 248 value-guarded branches.

249 **Multi-condition guards.** The formalism shall
 250 combine conditions of different kinds (event happen-
 251 ings, value comparisons, occurrence counts, historical
 252 values) into a single guard. *Example*: “If e_A occurs
 253 and $B > 100$ and e_C has occurred at least twice, then
 254 D must follow.” *Solution*: $\text{AllOf}(C)$ takes a finite set
 255 of predicates of any kind and forms their conjunction;
 256 the formalism’s predicates form a uniform space **Pred**,
 257 so event-, value-, count-, and history-based predicates
 258 can all be conjoined.

259 **Disjunctive triggers.** The formalism shall express
 260 a consequence that follows from any one of several
 261 alternative triggers. *Example*: “If e_A or e_B or e_C occurs,
 262 then D must hold.” *Solution*: $\text{AnyOf}(C)$ forms the
 263 disjunction of a finite set of predicates and may be
 264 used as the cause of a *Causes* construct.

Set-valued quantification. The formalism shall 265
 266 quantify over a finite set of entities and reason about
 267 properties of the subset that satisfies a predicate. *Ex-*
 268 *ample*: “For every entity in $\{A, B, C, D\}$, predicate P
 269 holds”; or “at most two entities in $\{A, B, C, D\}$ have
 270 a value greater than 10.” *Solution*: The Set entity
 271 kind together with the quantifiers $\text{ForAll}(S, P)$ and
 272 $\text{Exists}(S, P)$, plus *Filter* and *Size* for cardinality rea-
 273 soning over the subset that satisfies a predicate.

Temporal patterns. Sequencing. The formalism 274
 275 shall require multiple actions to occur in a specified
 276 order. *Example*: “ e_A then e_B then e_C .” *Solution*:
 277 $\text{Sequence}(\psi_1, \dots, \psi_n) \Leftrightarrow \exists t_1 \leq t_2 \leq \dots \leq t_n : \psi_i(\rho, t_i)$ for all i .

Time-bounded causation. The formalism shall re- 278
 279 quire an effect to occur within a bounded duration of a
 280 cause. *Example*: “If e_A occurs, e_B must occur within 2
 281 seconds.” *Solution*: $\text{CausesWithin}(\psi_{\text{cond}}, \psi_{\text{effect}}, d) \Leftrightarrow$
 282 $\forall t : \psi_{\text{cond}}(\rho, t) \Rightarrow \exists t' \in [t, t + d] : \psi_{\text{effect}}(\rho, t')$.

Sliding temporal windows. The formalism shall 283
 284 reason about a window of fixed length ending at the
 285 current time. *Example*: “ e_A occurred at least twice
 286 in the last 10 seconds.” *Solution*: An interval con-
 287 structor applied to *Now* produces the window, e.g.
 288 $\text{MkIntervalOO}(\text{Now} - d, \text{Now})$, which can then be passed
 289 to EvtOccCount_ρ or used as an argument to a *ForAll/Exists*
 290 over time.

Existence and global constraints. The formalism 291
 292 shall express both “at some point in the trace” and “at
 293 every time point in the trace”. *Example*: “Eventually A
 294 equals 5”; “ A is always non-negative”. *Solution*: The
 295 temporal constructs $\text{Eventually}(\psi) \Leftrightarrow \exists t : \psi(\rho, t)$ and
 296 $\text{Always}(\psi) \Leftrightarrow \forall t : \psi(\rho, t)$.

3.2 Formalism design decisions 297

3.3 Formalism definition 298

3.4 Known limitations and possible future ex- 299 tensions 300

[[Although this work claims that the formalism 301
 302 should be close to english and is close in the sense
 303 that it uses english words, the words themselves
 304 might not appear in a similar order than they would
 305 in english. This might be a limitation.]]

3.5 Compatibility with well-known formal spec- 306 ification methods 307

3.6 Requirement and test case examples 308

3.7 Future work 309

4. Conclusions 310

Acknowledgements 311

I would like to thank my supervisor Ing. Aleš Smrčka 312
 313 Ph.D. for his help.

314 References

- 315 [1] BEG, A., O'DONOGHUE, D. and MONAHAN, R.
316 *A Short Survey on Formalising Software Require-*
317 *ments with Large Language Models.* 2025. Avail-
318 able at: [https://arxiv.org/abs/2506.](https://arxiv.org/abs/2506.11874)
319 [11874](https://arxiv.org/abs/2506.11874).
- 320 [2] COSLER, M., HAHN, C., MENDOZA, D.,
321 SCHMITT, F. and TRIPPEL, C. *Nl2spec: Inter-*
322 *actively Translating Unstructured Natural Lan-*
323 *guage to Temporal Logics with Large Language*
324 *Models.* 2023. Available at: [https://arxiv.](https://arxiv.org/abs/2303.04864)
325 [org/abs/2303.04864](https://arxiv.org/abs/2303.04864).
- 326 [3] FANG, W., LI, M., LI, M., YAN, Z., LIU, S.
327 et al. AssertLLM: Generating Hardware Verifi-
328 cation Assertions from Design Specifications via
329 Multi-LLMs. In: *2024 IEEE LLM Aided Design*
330 *Workshop (LAD).* 2024, p. 1–1.
- 331 [4] FAZELNIA, M., MIRAKHORLI, M.
332 and BAGHERI, H. Translation Titans, Reasoning
333 Challenges: Satisfiability-Aided Language
334 Models for Detecting Conflicting Requirements.
335 In: *Proceedings of the 39th IEEE/ACM Inter-*
336 *national Conference on Automated Software*
337 *Engineering (ASE '24).* New York, NY, USA:
338 Association for Computing Machinery, 2024,
339 p. 2294–2298.
- 340 [5] GHOSH, S., ELENIOUS, D., LI, W., LINCOLN,
341 P., SHANKAR, N. et al. ARSENAL: Auto-
342 matic Requirements Specification Extraction
343 from Natural Language. In: RAYADURGAM, S.
344 and TKACHUK, O., ed. *NASA Formal Methods.*
345 Cham: Springer International Publishing, 2016,
346 p. 41–46.
- 347 [6] GIANNAKOPOULOU, D., MAVRIDOU, A.,
348 RHEIN, J., PRESSBURGER, T., SCHUMANN, J.
349 et al. Formal Requirements Elicitation with
350 FRET. In: *Joint Proceedings of REFSQ-2020*
351 *Workshops, Doctoral Symposium, Live Studies*
352 *Track, and Poster Track.* Pisa, Italy: [b.n.], 2020.
- 353 [7] JIANG, A. Q., LI, W., TWORKOWSKI, S.,
354 CZECHOWSKI, K., ODRZYGÓŹDŹ, T. et al. Thor:
355 Wielding Hammers to Integrate Language Mod-
356 els and Automated Theorem Provers. In: *Ad-*
357 *vances in Neural Information Processing Sys-*
358 *tems.* Curran Associates, Inc., 2022, p. 8360–
359 8373.
- 360 [8] NAYAK, A., TIMMAPATHINI, H. P., MURALI,
361 V., PONNALAGU, K., VENKOPARAO, V. G.
362 et al. Req2Spec: Transforming Software Re-
363 quirements into Formal Specifications Using Nat-
ural Language Processing. In: *Requirements*
364 *Engineering: Foundation for Software Quality*
365 *(REFSQ 2022).* Berlin, Heidelberg: Springer-
366 Verlag, 2022, p. 87–95.
367
- 368 [9] NOWAKOWSKI, W., ŚMIAŁEK, M., AM-
369 BROZIEWICZ, A. and STRASZAK, T. Re-
370 quirements-Level Language and Tools
371 for Capturing Software System Essence. *Com-*
372 *puter Science and Information Systems.* 2013,
373 vol. 10, no. 4, p. 1499–1524.
- 374 [10] QUAN, X., VALENTINO, M., DENNIS, L. A.
375 and FREITAS, A. *Verification and Refinement*
376 *of Natural Language Explanations through LLM-*
377 *Symbolic Theorem Proving.* 2024. Available at:
378 <https://arxiv.org/abs/2405.01379>.
- 379 [11] YANG, K., SWOPE, A., GU, A., CHALAMALA,
380 R., SONG, P. et al. LeanDojo: Theorem Prov-
381 ing with Retrieval-Augmented Language Models.
382 In: *Advances in Neural Information Processing*
383 *Systems.* Curran Associates, Inc., 2023, p. 21573–
384 21612.